SQL Basics

SQL (Structured Query Language) is the standard language for working with relational databases. A relational database stores data in tables made of rows (records) and columns (fields).

The examples use two tables:

students			
id	**name**	**age**	**course_id**
1	Sara	20	10
2	Leah	22	20
3	Rivka	21	NULL

courses	
id	**title**
10	Python
20	Java
30	SQL

SELECT

SELECT reads data from a table. * means all columns.

```
SELECT * FROM students;
SELECT name, age FROM students;
SELECT DISTINCT age FROM students;   -- unique values only
```

WHERE

WHERE filters rows by a condition.

```
SELECT name FROM students WHERE age > 20;
SELECT * FROM students WHERE name LIKE 'S%';     -- names starting with S
SELECT * FROM students WHERE age BETWEEN 20 AND 21;
SELECT * FROM students WHERE course_id IS NULL;  -- never use = NULL
```

ORDER BY and LIMIT

```
SELECT * FROM students ORDER BY age DESC LIMIT 2;
```

ASC sorts ascending (the default), DESC sorts descending.

JOIN

A JOIN combines rows from two tables based on a related column.

INNER JOIN returns only rows that have a match in both tables. Rivka (course_id NULL) and the SQL course (no students) are not returned.

```
SELECT s.name, c.title
FROM students s
INNER JOIN courses c ON s.course_id = c.id;
```

LEFT JOIN returns all rows from the left table, and the matching rows from the right table. When there is no match, the right side columns are NULL. Here Rivka is returned with title NULL.

```
SELECT s.name, c.title
FROM students s
LEFT JOIN courses c ON s.course_id = c.id;
```

RIGHT JOIN is the mirror of LEFT JOIN: all rows from the right table. **FULL OUTER JOIN** returns all rows from both tables.

Aggregate Functions and GROUP BY

Aggregate functions compute one value from many rows: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.

`GROUP BY` splits rows into groups and runs the aggregate function on each group.

```
SELECT course_id, COUNT(*) AS num_students, AVG(age) AS avg_age
FROM students
GROUP BY course_id;
```

WHERE vs HAVING: `WHERE` filters rows before grouping. `HAVING` filters groups after grouping, so it can use aggregate functions.

```
SELECT course_id, COUNT(*)
FROM students
GROUP BY course_id
HAVING COUNT(*) > 1;
```

Changing Data

```
INSERT INTO students (id, name, age, course_id) VALUES (4, 'Miriam', 23, 30);
UPDATE students SET age = 21 WHERE id = 1;
DELETE FROM students WHERE id = 4;
```

Warning: `UPDATE` or `DELETE` without `WHERE` changes every row in the table.

Keys

A **primary key** uniquely identifies each row. It cannot be NULL or repeat. A **foreign key** is a column that points to the primary key of another table, like `students.course_id` pointing to `courses.id`. Foreign keys keep data consistent between tables.

```
CREATE TABLE courses (
    id INT PRIMARY KEY,
    title VARCHAR(100) NOT NULL
);

CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    course_id INT,
    FOREIGN KEY (course_id) REFERENCES courses(id)
);
```

Order of Execution

SQL clauses run in this order, which is different from the order they are written: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. This is why an alias defined in SELECT cannot be used in WHERE.